

K49 Character Classification

Avaliação de Modelos de Deep Learning

Miguel Grilo (58387)

Jorge Couto (58656)

Licenciatura em Inteligência Artificial e Ciência de Dados
Universidade de Évora

Junho de 2026

Resumo

Este projeto explora e compara 11 arquiteturas distintas de *Deep Learning* aplicadas à classificação dos 49 caracteres japoneses do dataset K49. A abordagem segue uma progressão desde Redes Neurais Artificiais (ANN) totalmente conectadas, passando por redes convolucionais e técnicas de regularização avançadas, até soluções de *Transfer Learning* e *Ensemble Learning*. Ao longo do trabalho, são analisados o impacto da profundidade das redes, da preservação da estrutura espacial dos dados, da resolução de entrada e de técnicas como *MixUp*, *Dropout* e *Batch Normalization* na capacidade de generalização dos modelos. Os resultados demonstram uma melhoria consistente de desempenho ao longo das arquiteturas testadas, culminando em modelos de estado da arte, com destaque para o YOLOv8-cls como melhor modelo individual, atingindo os melhores valores de *accuracy* e *F1-score*. Adicionalmente, a abordagem de *ensemble learning* revelou-se eficaz na redução de variância e aumento da robustez do sistema.

Conteúdo

1	Introdução	4
2	Fundamentação Teórica	4
2.1	Redes Neurais Artificiais	4
2.2	Classificação de Imagens	4
2.3	Funções de Ativação	5
2.4	Funções de Perda	5
2.5	Otimizadores	5
3	Arquitetura e Engenharia do Projeto	5
3.1	Estrutura do Projeto	5
3.2	Fluxo de Execução	6
3.3	Organização dos Diretórios	6
3.4	Tecnologias Utilizadas	6
4	Abordagens de Modelação e Avaliação	7
4.1	Classifier 1: Simple ANN	7
4.1.1	Descrição e Arquitetura:	7
4.1.2	Iterações:	7
4.1.3	Resultados:	7
4.1.4	Análise:	7
4.2	Classifier 2: Deep ANN	7
4.2.1	Descrição e Arquitetura:	7
4.2.2	Iterações:	8
4.2.3	Resultados:	8
4.2.4	Análise:	8
4.3	Classifier 3: Advanced CNN	8
4.3.1	Descrição e Arquitetura:	8
4.3.2	Iterações:	8
4.3.3	Resultados:	9
4.3.4	Análise:	9
4.4	Classifier 4: PyTorch ANN	9
4.4.1	Descrição e Arquitetura:	9
4.4.2	Iterações:	9
4.4.3	Resultados:	9
4.4.4	Análise:	10
4.5	Classifier 5: ANN Ensemble Learning	10
4.5.1	Descrição e Arquitetura:	10
4.5.2	Iterações:	10
4.5.3	Resultados:	10
4.5.4	Análise:	10
4.6	Classifier 6: Transfer Learning (ResNet50V2)	11
4.6.1	Descrição e Arquitetura:	11
4.6.2	Iterações:	11
4.6.3	Resultados	11
4.6.4	Análise:	11
4.7	Classifier 7: Hyperparameter Tuning (KerasTuner)	11

4.7.1	Descrição e Arquitetura:	11
4.7.2	Iterações:	12
4.7.3	Resultados:	12
4.7.4	Análise:	12
4.8	Classifier 8: Residual CNN (Skip Connections)	12
4.8.1	Descrição e Arquitetura:	12
4.8.2	Iterações:	13
4.8.3	Resultados:	13
4.8.4	Análise:	13
4.9	Classifier 9: CNN com MixUp Augmentation	13
4.9.1	Descrição e Arquitetura:	13
4.9.2	Iterações:	14
4.9.3	Resultados:	14
4.9.4	Análise:	14
4.10	Classifier 10: Transfer Learning (YOLOv8-cla)	14
4.10.1	Descrição e Arquitetura:	14
4.10.2	Iterações:	14
4.10.3	Resultados:	15
4.10.4	Análise:	15
4.11	Classifier 11: Mixed Ensemble Learning	15
4.11.1	Descrição e Arquitetura:	15
4.11.2	Iterações:	15
4.11.3	Resultados:	16
4.11.4	Análise:	16
5	Desafios Técnicos e Otimização Computacional	16
6	Conclusões	16

1 Introdução

No âmbito da unidade curricular de Redes Neurais Artificiais foi proposto o desenvolvimento de um projeto dedicado ao treino de uma rede neuronal para classificação de imagens, recorrendo a um dos conjuntos de dados sugeridos. Para a realização deste trabalho, em vez do dataset *Kuzushiji-MNIST* de 10 classes padrão, optou-se pela utilização da sua variante avançada e estendida: o dataset *Kuzushiji-49* (K49). Esta versão, constituída por 49 classes de caracteres japoneses manuscritos, foi selecionada propositadamente para elevar a complexidade da tarefa, introduzir um desafio acrescido devido ao forte desequilíbrio e semelhança morfológica entre classes, e testar de forma rigorosa a capacidade de generalização das arquiteturas desenvolvidas.

Embora o objetivo inicial consistisse no desenvolvimento de uma única rede neuronal, optou-se por explorar diferentes arquiteturas e estratégias de otimização, permitindo comparar o desempenho de diversas abordagens na resolução do mesmo problema. Ao longo do projeto foram implementados modelos de complexidade crescente, desde Redes Neurais Artificiais (Artificial Neural Networks – ANN) tradicionais até Redes Neurais Convolucionais (Convolutional Neural Networks – CNN), técnicas de *Transfer Learning*, métodos de *Ensemble Learning* e otimização automática de hiperparâmetros.

O presente relatório descreve a metodologia adotada, a organização do projeto, os modelos desenvolvidos e os resultados experimentais obtidos, apresentando uma análise comparativa das diferentes abordagens e identificando as soluções que demonstraram melhor desempenho na classificação do dataset considerado.

2 Fundamentação Teórica

2.1 Redes Neurais Artificiais

As Redes Neurais Artificiais (Artificial Neural Networks – ANN) são modelos computacionais inspirados na estrutura e funcionamento do cérebro humano. São constituídas por um conjunto de neurónios artificiais organizados em camadas interligadas, responsáveis pelo processamento de informação através da propagação e transformação dos dados de entrada[2]. A capacidade de aprendizagem destes modelos resulta do ajuste iterativo dos pesos associados às ligações entre neurónios durante o processo de treino. A flexibilidade na definição da arquitetura da rede, nomeadamente no número de camadas, de neurónios e de ligações existentes, permite adaptar estes modelos a uma grande variedade de problemas de classificação, regressão e reconhecimento de padrões. Por este motivo, as Redes Neurais Artificiais constituem atualmente uma das principais ferramentas utilizadas nas áreas de *Machine Learning* e *Deep Learning*.

2.2 Classificação de Imagens

A Classificação de Imagens consiste na utilização de modelos de *Machine Learning* para atribuir automaticamente uma classe a uma imagem, com base no seu conteúdo visual. Para tal, as imagens são representadas sob a forma de matrizes de píxeis, cujos valores correspondem à intensidade das componentes de cor. Antes do processo de treino é comum aplicar técnicas de pré-processamento, como a normalização dos valores dos píxeis, de forma a facilitar a aprendizagem do modelo. Durante o treino, a Rede Neuronal recebe um conjunto de imagens previamente classificadas, ajustando iterativamente os

seus parâmetros internos de modo a aprender padrões representativos de cada classe. Após concluído este processo, o modelo torna-se capaz de generalizar o conhecimento adquirido e classificar corretamente novas imagens que não fizeram parte do conjunto de treino[1].

2.3 Funções de Ativação

As funções de ativação introduzem não linearidade nas Redes Neurais Artificiais, permitindo-lhes aprender relações complexas entre os dados de entrada e as respectivas saídas. Sem estas funções, uma rede composta por várias camadas seria equivalente a um único modelo linear, limitando significativamente a sua capacidade de aprendizagem[1]. Existem diversas funções de ativação, sendo algumas das mais utilizadas a *Rectified Linear Unit* (ReLU), a *Sigmoid* e a *Hyperbolic Tangent* (Tanh). A escolha da função de ativação depende da arquitetura da rede e da natureza do problema, podendo influenciar diretamente a velocidade de convergência e o desempenho do modelo.

2.4 Funções de Perda

A função de perda (*Loss Function*) constitui uma métrica utilizada para avaliar o desempenho da Rede Neuronal durante o processo de treino. Esta função compara as previsões produzidas pelo modelo com os valores reais esperados, produzindo um valor que representa o erro cometido[1]. O objetivo do treino consiste em minimizar este erro através do ajuste iterativo dos pesos da rede. Em problemas de classificação, uma das funções de perda mais utilizadas é a *Categorical Cross-Entropy*, por permitir avaliar eficazmente a discrepância entre as probabilidades previstas pelo modelo e as classes corretas.

2.5 Otimizadores

Os otimizadores são algoritmos responsáveis pela atualização dos pesos da Rede Neuronal durante o processo de treino. A partir da informação obtida pela função de perda e dos gradientes calculados através do algoritmo de *Backpropagation*, estes determinam a direção e a magnitude das alterações necessárias para reduzir progressivamente o erro do modelo[1]. Entre os algoritmos mais utilizados destacam-se o *Stochastic Gradient Descent* (SGD), o *RMSprop* e o *Adaptive Moment Estimation* (Adam), sendo este último amplamente adotado devido à sua rápida convergência e à capacidade de adaptar automaticamente a taxa de aprendizagem ao longo do treino.

3 Arquitetura e Engenharia do Projeto

3.1 Estrutura do Projeto

O projeto encontra-se organizado de forma modular, permitindo separar claramente os diferentes componentes responsáveis pelo desenvolvimento, treino e avaliação dos modelos. A arquitetura adotada segue uma abordagem orientada à experimentação, na qual múltiplas implementações de modelos são geridas de forma centralizada através de um único ponto de execução. O ficheiro principal `main.py` atua como o ponto de entrada do sistema, sendo responsável pela coordenação de todo o pipeline experimental, incluindo a

preparação dos dados, seleção dos modelos a executar e geração automática dos relatórios de desempenho.

3.2 Fluxo de Execução

A execução do projeto é realizada através do ficheiro `main.py`, que permite seleccionar dinamicamente o modelo ou conjunto de modelos a treinar através de argumentos de linha de comandos. O pipeline inicia-se com a configuração do ambiente de execução, incluindo a deteção automática do hardware disponível (CPU ou GPU) e a inicialização das bibliotecas necessárias, nomeadamente Keras e PyTorch. Em seguida, os dados do dataset K49 são carregados e preparados para treino. Posteriormente, o sistema percorre um registo centralizado de modelos (*MODELS_REGISTRY*), onde cada arquitetura está associada ao seu respetivo módulo de execução. Cada modelo é então treinado ou avaliado de forma independente, sendo as métricas de desempenho (accuracy, precision, recall e **F1-score**) armazenadas para análise comparativa.

No final da execução, é gerado um resumo global dos resultados e exportados relatórios detalhados, incluindo matrizes de confusão e métricas por modelo.

3.3 Organização dos Diretórios

A estrutura do projeto encontra-se dividida em diferentes diretórios, cada um com responsabilidades bem definidas. O diretório `dataset/` contém os ficheiros originais do conjunto de dados K49, bem como uma versão convertida para o formato exigido por modelos de *Transfer Learning* baseados em YOLO. O diretório `functions/` constitui o núcleo funcional do projeto, contendo as implementações reutilizáveis associadas a redes neurais, otimização, carregamento de dados e utilitários de suporte ao treino. O diretório `versions/` contém as implementações individuais de cada modelo testado, refletindo diferentes abordagens experimentais e configurações de hiperparâmetros utilizadas ao longo do projeto. Os resultados obtidos são armazenados no diretório `results/`, enquanto os pesos dos modelos treinados são guardados em `weights/`. Diretórios adicionais como `runs/` e `keras_tuner/` são gerados automaticamente durante a execução de processos de treino e otimização.

3.4 Tecnologias Utilizadas

O desenvolvimento do projeto foi realizado em Python, recorrendo a um conjunto de bibliotecas orientadas para *Machine Learning* e *Deep Learning*. Entre as principais ferramentas utilizadas destacam-se TensorFlow/Keras e PyTorch, utilizadas para a implementação e treino das redes neurais desenvolvidas. Para modelos baseados em *Transfer Learning*, foi utilizada a biblioteca Ultralytics YOLOv8, permitindo a exploração de arquiteturas de estado da arte em tarefas de classificação de imagens. A gestão de dependências e do ambiente de execução foi realizada através da ferramenta `uv`, garantindo reprodutibilidade e consistência entre diferentes sistemas. Adicionalmente, foram utilizadas bibliotecas auxiliares para processamento de dados, visualização de resultados e geração automática de relatórios.

Nota: O código completo, juntamente com instruções detalhadas de instalação e replicação do ambiente virtual através da ferramenta `uv`, encontra-se documentado no `README.md` do repositório oficial do projeto.

4 Abordagens de Modelação e Avaliação

Ao longo deste projeto, foram desenvolvidas e avaliadas 11 abordagens distintas, iterando sobre a complexidade arquitetural e métodos de regularização.

4.1 Classifier 1: Simple ANN

4.1.1 Descrição e Arquitetura:

O modelo fundamental de partida. Consiste numa Rede Neuronal Artificial básica para mapear a capacidade de aprendizagem inicial. A arquitetura deste modelo base é composta por uma camada de entrada que aplica um *Flatten* (achatamento) à imagem original de 28x28 píxeis, convertendo a matriz 2D num vetor unidimensional de 784 elementos. Segue-se uma camada densa (*Fully Connected*) com 128 (e posteriormente 256) neurónios utilizando a função de ativação ReLU. A escolha da ReLU justifica-se pela sua eficiência computacional e capacidade de mitigar o desaparecimento do gradiente. Para prevenir a memorização dos dados de treino (*overfitting*), foi introduzida uma camada de *Dropout* com uma taxa de 0.2, que desativa aleatoriamente 20% dos neurónios durante o treino. A camada de saída contém 49 neurónios com ativação *Softmax*, gerando uma distribuição de probabilidades para as 49 classes do dataset K49.

4.1.2 Iterações:

A Iteração 1 serviu de baseline (128 neurónios, 5 epochs), enquanto a Iteração 2 otimizou a capacidade para 256 neurónios e 15 epochs.

4.1.3 Resultados:

Iteração	Accuracy	Precision	Recall	F1-Score
1 (Baseline)	74.25%	75.51%	74.25%	74.56%
2 (Optimized)	81.43%	82.03%	81.43%	81.53%

Tabela 1: Métricas de avaliação para o Simple ANN.

4.1.4 Análise:

Estabeleceu uma base sólida, mas mostrou limitações devido à ausência de extração de características espaciais (flattening direto da imagem).

4.2 Classifier 2: Deep ANN

4.2.1 Descrição e Arquitetura:

O *Deep ANN* expande a arquitetura base através da introdução de múltiplas camadas ocultas, permitindo a aprendizagem de representações mais complexas. Estruturalmente, a rede aplica um *Flatten* inicial e distribui-se por duas camadas densas sequenciais. A principal inovação estrutural justifica-se pela introdução de *Batch Normalization* imediatamente após cada uma das camadas densas ocultas, o que normaliza as ativações da

camada anterior, acelera a convergência e confere maior estabilidade ao processo de otimização. Para mitigar o risco elevado de sobreajuste (*overfitting*) decorrente do incremento substancial no número de parâmetros, aplicou-se uma taxa de *Dropout* mais severa de 0.3 após cada bloco funcional. A rede é otimizada através do algoritmo *Adam*, minimizando a função de perda de entropia cruzada e mapeando a camada de saída para 49 neurónios via *Softmax*.

4.2.2 Iterações:

Foram testadas duas configurações: uma versão inicial com 300 e 100 neurónios nas camadas ocultas, e uma versão otimizada com 512 e 128 neurónios, respetivamente. Foi também aplicado *early stopping* para evitar sobreajuste.

4.2.3 Resultados:

Iteração	Accuracy	Precision	Recall	F1-Score
1 (Baseline)	81.93%	82.49%	81.93%	82.07%
2 (Optimized)	84.90%	85.80%	84.90%	85.13%

Tabela 2: Resultados do Deep ANN

4.2.4 Análise:

O aumento da profundidade melhora consistentemente o desempenho, evidenciando a vantagem de arquiteturas mais expressivas na classificação do K49.

4.3 Classifier 3: Advanced CNN

4.3.1 Descrição e Arquitetura:

O *Advanced CNN* introduz a primeira abordagem convolucional profunda do trabalho, explorando filtros para extração de padrões espaciais como traços e curvas nos caracteres do dataset K49. Ao contrário das abordagens lineares anteriores, esta arquitetura foi desenhada especificamente para preservar a estrutura bidimensional das imagens. O modelo é construído de forma modular através de blocos compostos por camadas convolucionais (*Conv2D*) com filtros de 3×3 (escalando de 32 a 128 filtros), funções de ativação ReLU e camadas de agrupamento máximo (*MaxPooling2D*) com janelas de 2×2 . A escolha de kernels 3×3 justifica-se pela eficácia na extração de primitivas geométricas locais (como os traços finos e curvas da caligrafia japonesa), enquanto o *MaxPooling* reduz a dimensionalidade e garante invariância a pequenas translações do caractere. Após o achatamento (*Flatten*) das matrizes de características, a cabeça de classificação emprega uma camada densa de até 256 neurónios, fortemente protegida contra o sobreajuste (*overfitting*) através de penalização por regularização L2 (1×10^{-4}), *Batch Normalization* e uma taxa de *Dropout* de 0.4.

4.3.2 Iterações:

O modelo foi evoluindo de forma incremental com aumento de profundidade, *data augmentation*, normalização, regularização L2 e ajuste dinâmico da taxa de aprendizagem.

4.3.3 Resultados:

Iteração	Accuracy	Precision	Recall	F1-Score
1 (Baseline)	90.56%	90.57%	90.56%	90.48%
2 (Dense=256)	91.71%	92.09%	91.71%	91.81%
3 (Deep+Aug+Reg)	90.88%	91.42%	90.88%	90.99%
4 (LR Scheduler)	93.84%	94.59%	93.84%	94.15%
5 (150 Epochs)	94.27%	94.79%	94.27%	94.46%

Tabela 3: Resultados do Advanced CNN

4.3.4 Análise:

As melhorias incrementais demonstram que regularização e otimização do treino têm impacto mais significativo do que apenas aumentar a complexidade do modelo.

4.4 Classifier 4: PyTorch ANN

4.4.1 Descrição e Arquitetura:

O *PyTorch ANN* reproduz a arquitetura do *Simple ANN* utilizando o framework PyTorch, com o propósito de avaliar a consistência de desempenho e comparar a flexibilidade do desenvolvimento de pipelines de baixo nível face às abstrações do Keras. A rede é estruturada mapeando uma camada linear de entrada (*nn.Linear*) que recebe o vetor achatado de 784 píxeis e o projeta para uma camada oculta totalmente conectada com ativação ReLU. Uma particularidade estrutural crítica nesta implementação reside na utilização da inicialização de pesos *Kaiming Uniform (He initialization)*, uma estratégia matematicamente otimizada para camadas que precedem ativações não-lineares baseadas em ReLU, garantindo que a variância dos sinais se mantenha estável e prevenindo o desaparecimento precoce dos gradientes. Ao contrário do ecossistema Keras, o fluxo de dados (*forward pass*), o carregamento em lotes via *DataLoaders*, o ciclo explícito de retropropagação e a atualização de pesos foram codificados manualmente. A camada final projeta os resultados diretamente para 49 neurónios, sendo convertida em distribuição probabilística por uma função *Softmax* na fase de inferência.

4.4.2 Iterações:

Foram testadas duas configurações: uma versão base com otimizador SGD e uma versão otimizada com Adam, 256 neurónios na camada oculta e 15 épocas de treino.

4.4.3 Resultados:

Iteração	Accuracy	Precision	Recall	F1-Score
1 (Baseline - SGD)	72.63%	74.52%	72.63%	73.20%
2 (Optimized - Adam)	80.66%	81.17%	80.66%	80.71%

Tabela 4: Resultados do PyTorch ANN

4.4.4 Análise:

O desempenho é semelhante ao Simple ANN, confirmando que o framework não é o fator determinante, mas sim a arquitetura e capacidade do modelo.

4.5 Classifier 5: ANN Ensemble Learning

4.5.1 Descrição e Arquitetura:

O *ANN Ensemble Learning* combina três redes neurais independentes com o objetivo de reduzir a variância das previsões e melhorar a robustez global do sistema através de *softmax averaging*. Em vez de depender de uma única arquitetura, o sistema agrupa três redes *Deep ANN* construídas com topologias propositadamente divergentes: um modelo com hiperparâmetros otimizados de base (Member A), um segundo modelo com uma taxa de *Dropout* elevada (0.4) para forçar a rede a aprender representações de características alternativas (Member B), e um terceiro modelo com um estrangulamento arquitetural na camada oculta, passando de 400 para 200 neurónios (Member C). Cada modelo é treinado separadamente, o que permite capturar diferentes padrões de erro. A justificação estatística desta topologia baseia-se no princípio do *Wisdom of the Crowd*: ao calcular a média matemática das distribuições de probabilidade geradas pelas três saídas *Softmax*, o *Ensemble* anula os enviesamentos individuais de cada rede e reduz drasticamente a variância do erro.

4.5.2 Iterações:

Foram treinados três modelos distintos: um modelo base otimizado (Member A), um modelo com maior regularização para reduzir overfitting (Member B) e um modelo com maior capacidade representacional através de camadas mais largas (Member C). Após o treino individual, as previsões são combinadas por média das probabilidades, formando o ensemble final.

4.5.3 Resultados:

Modelo	Accuracy	Precision	Recall	F1-Score
Member A	84.36%	–	–	84.65%
Member B	83.18%	–	–	83.43%
Member C	84.74%	–	–	84.92%
Ensemble Final	85.91%	87.01%	85.91%	86.27%

Tabela 5: Resultados do ANN Ensemble Learning

4.5.4 Análise:

O ensemble melhora a estabilidade e consistência das previsões, demonstrando que a combinação de modelos com diferentes enviesamentos e capacidades representacionais permite obter melhor generalização do que modelos individuais.

4.6 Classifier 6: Transfer Learning (ResNet50V2)

4.6.1 Descrição e Arquitetura:

O modelo recorre à técnica de *Transfer Learning* utilizando a arquitetura *ResNet50V2* pré-treinada no dataset ImageNet, com o objetivo de explorar a transferência de representações visuais robustas para o domínio dos caracteres japoneses. Estruturalmente, o sistema importa os pesos convolucionais profundos da rede original. Contudo, como a ResNet executa múltiplas operações agudas de redução de dimensionalidade geométrica (*downsampling*), a injeção direta de imagens de 28×28 píxeis destruiria a informação espacial de traços finos antes mesmo de atingir as camadas de decisão. Para contornar esta limitação arquitetural, justificou-se a implementação de um pré-processamento incorporado no próprio grafo do modelo: uma camada de *Resizing* dinâmico seguida da concatenação do canal único de tons de cinzento para gerar três canais fictícios (simulando o formato RGB exigido). A saída da base pré-treinada foi ligada a uma camada de *Global Pooling*, alimentando uma nova cabeça de classificação densa (256 neurónios) com forte regularização *Dropout* (0.5) para evitar a memorização cega dos dados de treino.

4.6.2 Iterações:

Foram testadas três fases: modelo congelado, fine-tuning parcial e configuração otimizada com aumento de resolução e *data augmentation*.

4.6.3 Resultados

Iteração	Accuracy	Precision	Recall	F1-Score
1 (Frozen Base)	54.87%	57.00%	54.87%	55.44%
2 (Fine-Tuning)	54.31%	60.09%	54.31%	54.90%
3 (SOTA Setup)	93.07%	93.71%	93.07%	93.30%

Tabela 6: Resultados do modelo Transfer Learning com ResNet50V2

4.6.4 Análise:

O aumento de resolução e fine-tuning completo foram essenciais para desbloquear o desempenho do modelo, permitindo adaptação eficaz ao domínio do dataset.

4.7 Classifier 7: Hyperparameter Tuning (KerasTuner)

4.7.1 Descrição e Arquitetura:

O modelo recorre ao *Keras Tuner* para realizar otimização automática de hiperparâmetros, eliminando o enviesamento empírico na seleção manual de arquiteturas. Em vez de uma configuração estática fixada *a priori*, o sistema explora algoritmicamente um espaço de pesquisa estruturado, variando combinações de filtros convolucionais, unidades em camadas densas, taxas de *dropout* e *learning rate*. A justificação para esta abordagem é delegar na máquina a descoberta da topologia ótima para a complexidade geométrica do

K49. A arquitetura final validada pelo algoritmo como ideal consistiu num bloco principal de 64 filtros convolucionais acoplado a uma camada de 512 unidades densas. De forma reveladora, a estrutura selecionada demonstrou que calibrações precisas na taxa de regularização (*Dropout* oscilando entre 0.2 e 0.4) têm um impacto estatisticamente mais significativo na prevenção de memorização e sobreajuste (*overfitting*) do que o aumento agressivo da profundidade convolucional pura.

4.7.2 Iterações:

O processo de otimização foi conduzido através de duas estratégias de busca: uma pesquisa mais rápida (*fast search*) para exploração inicial do espaço de soluções e uma pesquisa mais aprofundada (*deep search*) para refinar as melhores configurações encontradas.

4.7.3 Resultados:

Iteração	Accuracy	Precision	Recall	F1-Score
1 (Fast Search)	89.79%	89.89%	89.79%	89.71%
2 (Deep Search)	90.10%	90.67%	90.10%	90.28%

Tabela 7: Resultados do modelo Keras Tuner

4.7.4 Análise:

A otimização automática dos hiperparâmetros conduz a ganhos de desempenho visíveis, demonstrando a eficácia da exploração algorítmica do espaço de soluções. Um fenómeno interessante observado ao cruzar as duas estratégias de busca foi a relação entre a profundidade da pesquisa e a regularização. Embora a *Fast Search* tenha estabelecido uma base competitiva muito rapidamente (89.79% de exatidão) ao adotar um Dropout mais agressivo (0.4), foi a pesquisa exaustiva da *Deep Search* que conseguiu extrair o desempenho máximo (90.10% de exatidão). Este resultado indica que, apesar de a *Deep Search* utilizar uma taxa de Dropout inferior (0.2), a combinação refinada de filtros convolucionais, unidades densas e uma taxa de aprendizagem mais conservadora permitiu ao modelo mitigar o sobreajuste e superiorizar-se na classificação de dados não vistos.

4.8 Classifier 8: Residual CNN (Skip Connections)

4.8.1 Descrição e Arquitetura:

O modelo introduz ligações residuais (*skip connections*) com o objetivo de melhorar o fluxo de gradiente e permitir o treino estável de arquiteturas mais profundas. Construída através da *Functional API* do Keras, esta arquitetura inova ao quebrar a sequencialidade rígida das redes tradicionais. O fluxo inicia-se com uma convolução base de 64 filtros (3×3) e *Batch Normalization*. À entrada do bloco residual, o mapa de características atual é preservado como um atalho (*shortcut*), enquanto o caminho principal processa duas convoluções adicionais de 64 filtros. Antes da aplicação da ativação ReLU final do bloco, a identidade original é somada diretamente ao mapa transformado através da operação *layers.Add()*. A justificação matemática para esta topologia reside na mitigação do

desaparecimento do gradiente (*Vanishing Gradient*), criando autoestradas que facilitam a retropropagação direta do erro e preservam os detalhes geométricos dos traços dos caracteres. O pipeline de classificação é finalizado com uma camada de *MaxPooling2D* (2×2), achatamento (*Flatten*), e uma camada densa de 256 neurónios protegida por um *Dropout* de 0.4 antes da camada *Softmax* de 49 classes.

4.8.2 Iterações:

A arquitetura base inclui blocos convolucionais com normalização por lotes e uma ligação residual entre camadas. Foram testadas duas configurações: uma versão com 20 épocas de treino e uma versão estendida com 40 épocas, de forma a avaliar o impacto de treino prolongado em redes com *skip connections*.

4.8.3 Resultados:

Iteração	Accuracy	Precision	Recall	F1-Score
1 (Baseline)	89.27%	89.96%	89.27%	89.49%
2 (40 Epochs)	90.54%	90.81%	90.54%	90.56%

Tabela 8: Resultados do modelo Residual CNN

4.8.4 Análise:

As ligações residuais melhoram a estabilidade do treino e permitem maior profundidade útil da rede, embora os ganhos observados sejam incrementais. O aumento do número de épocas introduz melhorias ligeiras, mas também sinais de sobreajuste, sugerindo necessidade de regularização adicional em treinos mais longos.

4.9 Classifier 9: CNN com MixUp Augmentation

4.9.1 Descrição e Arquitetura:

O modelo introduz uma mudança de paradigma face aos anteriores: em vez de complexificar a arquitetura da rede, foca-se na manipulação estocástica do fluxo de dados através de *MixUp Augmentation*. Esta técnica combina linearmente pares de imagens aleatórias e os seus respetivos rótulos (*one-hot encoded*) durante o treino. A justificação teórica desta abordagem é forçar o modelo a aprender fronteiras de decisão lineares e suaves entre as 49 classes do K49, em vez de memorizar características absolutas. Para suportar esta aprendizagem, a arquitetura subjacente é uma CNN robusta e eficiente: dois blocos convolucionais (3×3) com 32 e 64 filtros, intercalados com *Batch Normalization* e *MaxPooling2D* (2×2). Como o *MixUp* gera "rótulos suaves" (distribuições de probabilidade em vez de classes inteiras), a função de perda foi obrigatoriamente adaptada para *Categorical Cross-Entropy*. O topo da rede consiste numa camada densa de 256 neurónios que, para além da regularização imposta pelo *MixUp*, é blindada contra o sobreajuste através de penalização L2 (1×10^{-4}), *Batch Normalization* e uma taxa de *Dropout* agressiva de 0.4.

4.9.2 Iterações:

A arquitetura CNN utilizada é relativamente simples, composta por camadas convolucionais seguidas de normalização por lotes, pooling e uma camada totalmente conectada com *dropout*. Foram testadas duas configurações de treino, com 40 e 80 épocas, para avaliar o impacto do treino prolongado sob forte regularização.

4.9.3 Resultados:

Iteração	Accuracy	Precision	Recall	F1-Score
1 (40 Epochs)	91.33%	92.29%	91.33%	91.68%
2 (80 Epochs)	91.35%	92.29%	91.35%	91.70%

Tabela 9: Resultados do modelo CNN com MixUp

4.9.4 Análise:

O *MixUp Augmentation* estabiliza o processo de treino e reduz significativamente o overfitting, conduzindo a uma convergência rápida do modelo. Como resultado, o aumento do número de épocas não produz melhorias relevantes, indicando que o modelo atinge um ponto de saturação de desempenho sob esta forma de regularização.

4.10 Classifier 10: Transfer Learning (YOLOv8-cls)

4.10.1 Descrição e Arquitetura:

O modelo *YOLOv8-cls* representa uma abordagem de *Transfer Learning* de estado da arte (SOTA), baseada na arquitetura *Ultralytics YOLOv8 Nano*, adaptada especificamente para tarefas de classificação de imagens. Ao contrário das abordagens anteriores (que processam matrizes na RAM através do Keras ou PyTorch), este modelo exigiu a conversão do dataset para uma estrutura hierárquica de diretórios no disco compatível com YOLO, permitindo o uso da pipeline interna otimizada de carregamento e *data augmentation* da framework Ultralytics. A arquitetura do YOLOv8 Nano assenta numa topologia extremamente profunda de blocos residuais desenhada para extração de características em múltiplas escalas. Devido a esta profundidade estrutural, justificou-se uma intervenção crítica na resolução de entrada: o redimensionamento progressivo das imagens do K49 desde os 28×28 originais até ao formato nativo do ImageNet (224×224 píxeis). Esta alteração estrutural foi imperativa para garantir que a rede tivesse área espacial suficiente para operar as sucessivas convoluções sem colapsar prematuramente as matrizes de características (*feature maps*) para 1×1 píxel antes de atingir as camadas de classificação lineares no topo da rede.

4.10.2 Iterações:

Foram testadas quatro configurações com variações de resolução e número de épocas. As duas primeiras iterações utilizam imagens de 32×32 (baseline), a terceira aumenta para 128×128 para melhorar a preservação de informação espacial, e a quarta utiliza 224×224

(*Overkill Mode*), explorando ao máximo o modelo pré-treinado.

4.10.3 Resultados:

Iteração	Accuracy	Precision	Recall	F1-Score
1 (10 Epochs, 32px)	80.48%	81.70%	80.48%	80.69%
2 (25 Epochs, 32px)	92.31%	93.18%	92.31%	92.57%
3 (15 Epochs, 128px)	96.23%	96.56%	96.23%	96.37%
4 (50 Epochs, 224px)	97.72%	97.80%	97.72%	97.85%

Tabela 10: Resultados do YOLOv8-cls

4.10.4 Análise:

O aumento da resolução teve impacto determinante no desempenho, demonstrando que arquiteturas profundas de *transfer learning* dependem fortemente da preservação da informação espacial na entrada.

4.11 Classifier 11: Mixed Ensemble Learning

4.11.1 Descrição e Arquitetura:

O modelo *Mixed Ensemble Learning* combina três arquiteturas distintas: uma *Advanced CNN*, um modelo *ResNet50V2* pré-treinado e uma CNN com *MixUp Data Augmentation*. O objetivo estrutural desta abordagem é explorar a complementaridade matemática entre modelos com vieses de aprendizagem e métodos de extração de características radicalmente heterogêneos. Esta meta-arquitetura opera de forma agnóstica a pesos próprios em tempo de inferência, atuando como uma camada de orquestração superior. Durante o fluxo de avaliação, cada imagem de teste é submetida em paralelo a três condutas de pré-processamento independentes (respeitando o formato exigido por cada classificador, como o *upscaling* tricanal para a ResNet e as matrizes originais em tons de cinzento para as CNNs). As distribuições de probabilidade brutas geradas nas respectivas camadas *Softmax* são então fundidas através de uma média matemática (*Softmax Averaging* ou *Soft Voting*). A justificação desta topologia assenta no princípio de que os espaços de erro de modelos com origens tão diversas não se sobrepõem, permitindo que os pontos fortes de um classificador anulem as vulnerabilidades e as incertezas dos restantes.

4.11.2 Iterações:

O modelo não envolve treino adicional, sendo composto por três redes previamente otimizadas. Na fase de inferência, cada imagem é processada pelos três modelos, respeitando os respetivos pré-processamentos, e as previsões são combinadas através da média das probabilidades (*softmax averaging*).

4.11.3 Resultados:

Modelo / Ensemble	Accuracy	Precision	Recall	F1-Score
Member 1 (Advanced CNN)	94.27%	94.79%	94.27%	94.46%
Member 2 (ResNet50V2)	93.07%	93.71%	93.07%	93.30%
Member 3 (MixUp CNN)	91.35%	92.29%	91.35%	91.70%
Ensemble Final	96.42%	96.50%	96.42%	96.60%

Tabela 11: Resultados do Mixed Ensemble Learning

4.11.4 Análise:

O ensemble demonstra elevada robustez e melhor desempenho global face aos modelos individuais, confirmando a eficácia da combinação de arquiteturas heterogêneas para reduzir variância e melhorar generalização.

5 Desafios Técnicos e Otimização Computacional

O desenvolvimento e execução simultânea de múltiplas arquiteturas de Deep Learning revelou diversas limitações técnicas associadas ao ambiente de computação utilizado, particularmente no que diz respeito à gestão de recursos GPU e compatibilidade entre frameworks. Todo o pipeline de treino e avaliação foi executado localmente utilizando uma GPU NVIDIA GeForce RTX 4050 de 6GB de VRAM. A limitação de memória física deste hardware tornou imperativa a adoção de estratégias dinâmicas de alocação de recursos. Uma das principais dificuldades esteve relacionada com a coexistência de diferentes ecossistemas de Deep Learning, nomeadamente TensorFlow/Keras, PyTorch e a framework Ultralytics YOLOv8. Estas bibliotecas apresentam estratégias distintas de gestão de memória GPU, o que resultou em conflitos ao nível da alocação de VRAM e da compatibilidade com bibliotecas CUDA e cuDNN. Para mitigar estes problemas, foi necessário implementar mecanismos explícitos de isolamento de ambiente. No caso do TensorFlow, foi utilizada configuração de *memory growth*, permitindo alocação dinâmica de memória GPU em vez de ocupação total imediata. Por outro lado, no caso do PyTorch, foi necessário remover variáveis de ambiente conflituosas relacionadas com bibliotecas do sistema, garantindo a utilização isolada das dependências instaladas via pip. Estas adaptações foram essenciais para permitir a execução conjunta de modelos baseados em Keras e PyTorch no mesmo sistema, evitando falhas de execução e conflitos de bibliotecas partilhadas. Apesar destas soluções, a execução de modelos de elevada complexidade, como YOLOv8-cls com imagens de alta resolução, manteve-se extremamente exigente do ponto de vista computacional, requerendo otimizações adicionais ao nível do batch size e da gestão de memória para garantir estabilidade durante o treino prolongado.

6 Conclusões

Este trabalho apresentou uma análise comparativa de diferentes abordagens de *Deep Learning* aplicadas à classificação de caracteres do dataset K49, abrangendo desde redes neuronais totalmente conectadas até arquiteturas de *Transfer Learning* e *Ensemble Learning*. Os resultados obtidos evidenciam uma evolução clara e consistente na capacidade

de generalização à medida que se aumenta a complexidade e sofisticação das arquiteturas. Em particular, os modelos baseados em *Artificial Neural Networks* apresentaram desempenhos mais limitados, o que reforça a dificuldade destas abordagens em capturar relações espaciais em dados de imagem. A introdução de redes convolucionais resultou num aumento significativo de desempenho, confirmando a importância da preservação da estrutura bidimensional dos dados em tarefas de visão computacional. Dentro das arquiteturas convolucionais, verificou-se que técnicas de regularização e aumento de dados desempenham um papel mais determinante na generalização do que o simples aumento da profundidade ou complexidade do modelo. Estratégias como *Dropout*, *Batch Normalization* e, em particular, *MixUp Data Augmentation*, mostraram-se fundamentais para reduzir o *overfitting* e estabilizar o processo de treino. Os modelos baseados em *Transfer Learning* destacaram-se como os mais performantes em termos absolutos, beneficiando do conhecimento previamente adquirido em grandes datasets como o ImageNet. Em particular, o YOLOv8-cls atingiu o melhor desempenho global, sendo decisivo o ajuste da resolução de entrada para 224x224, o que evitou a perda de informação espacial em camadas profundas da rede. Por outro lado, o modelo de *Mixed Ensemble Learning* demonstrou a eficácia da combinação de arquiteturas heterogêneas, permitindo reduzir a variância das previsões e aumentar a robustez do sistema. Contudo, observou-se que a inclusão de modelos significativamente mais fortes pode, em certos casos, introduzir um efeito de diluição quando é utilizada uma simples média de probabilidades. Globalmente, conclui-se que o YOLOv8-cls representa o melhor modelo em termos de desempenho absoluto, enquanto a *Advanced CNN* constitui a solução mais equilibrada entre performance, simplicidade e custo computacional. O ensemble, apesar de não ultrapassar o melhor modelo individual, demonstrou elevada robustez e utilidade em cenários onde a estabilidade das previsões é prioritária. Importa salientar que a arquitetura YOLOv8 foi intencionalmente excluída do Mixed Ensemble por dois motivos: primeiro, a sua framework baseada em disco é estruturalmente incompatível com a média dinâmica de probabilidades em RAM dos modelos Keras; segundo, o seu desempenho isolado (97.72%) é de tal forma superior que a média matemática com classificadores mais fracos iria atuar como um fator de diluição, penalizando o resultado final. Em síntese, este trabalho evidencia que o desempenho em tarefas de classificação de imagens não depende apenas da profundidade das redes neuronais, mas sim de um equilíbrio cuidadoso entre arquitetura, qualidade e pré-processamento dos dados, resolução de entrada e estratégias de regularização.

Referências

- [1] Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press (2016)
- [2] Haykin, S.: Neural Networks and Learning Machines. Pearson, 3 edn. (2009)